

Actividad 3

Alumno

Alan Lam Páez

Matrícula

T03063809

Materia

Gestión y Automatización de
Pruebas de Software

Maestro(a)

Cristopher Gerardo Gaytán Díaz

Correo

al03063809@tecmilenio.mx

Fecha

07/09/2026

Introducción

El objetivo de este ejercicio es implementar una prueba automatizada para el sitio web de GeekStore. Para esto, se utiliza un ambiente de pruebas en Windows 11 con Python, Pytest y Playwright, permitiendo automatizar la interacción con el navegador y verificar que el sitio funcione de acuerdo con las condiciones establecidas en las pruebas.

La prueba implementada actualmente comprueba que el navegador logre acceder correctamente al sitio GeekStore y que la página cargada corresponda a "index.php".

Tecnologías y herramientas utilizadas

Herramienta	Descripción
Python	Lenguaje de programación utilizado para desarrollar las pruebas automatizadas.
venv	Entorno virtual de Python utilizado para mantener aisladas las dependencias del proyecto.
Pytest	Framework utilizado para ejecutar las pruebas, organizar los casos de prueba y determinar si cada prueba pasa o falla.
Playwright	Herramienta utilizada para automatizar el navegador. Permite abrir Chromium, navegar por páginas web y realizar diferentes acciones sobre ellas.
Chromium	Navegador utilizado por Playwright para ejecutar las pruebas.

Visual Studio Code	Entorno de desarrollo utilizado para editar el código y ejecutar los comandos del proyecto.
Git	Para obtener el código fuente del proyecto.

Descripción del proyecto

El proyecto está diseñado para realizar pruebas automatizadas sobre el sitio web GeekStore.

Su estructura separa las diferentes responsabilidades del proyecto. La carpeta *tests* contiene los casos de prueba, mientras que *pages* contiene los Page Objects (clase de Python) utilizados para representar las páginas del sitio.

La estructura principal es:

config/	Configuración general y URL base
core/	Administración del navegador
pages/	Page Objects reutilizables
tests/	Casos de prueba
reports/	Reportes generados por las pruebas

Funcionamiento de la prueba

Flujo General y Ejecución

El proyecto utiliza el patrón Page Object Model (POM). Este patrón permite separar la lógica de las pruebas de la lógica utilizada para interactuar con las páginas web. La prueba principal implementada se encuentra en `test_login.py`, donde `LoginPage` representa la página utilizada para las operaciones relacionadas con el login.

Dentro de `login_page.py` se define la URL de GeekStore:

```
self.url = "https://testing.geekqa.net/public_html/public/index.php"
```

El flujo general de la prueba con test_login.py es:

1. Pytest inicia la ejecución de las pruebas automatizadas.
2. Pytest localiza y ejecuta el archivo test_login.py.
3. Dentro de este archivo, la clase TestLogin organiza la prueba relacionada con el inicio de sesión.
4. La prueba utiliza la clase LoginPage, que representa la página de inicio de sesión y centraliza las acciones disponibles en ella.
5. LoginPage ejecuta el método navigate() para abrir la página correspondiente de GeekStore.
6. Playwright controla el navegador y ejecuta las acciones definidas en la prueba.
7. Chromium se utiliza como navegador para cargar e interactuar con la aplicación.
8. La aplicación GeekStore se abre en el navegador.
9. Finalmente, la prueba verifica que la URL obtenida sea la esperada, confirmando que la navegación se realizó correctamente.

```
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Impossible\geekstore-automation-playwright\venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.13.2', 'Platform': 'Windows-11-10.0.22631-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'base-url': '2.1.0', 'html': '4.2.0', 'metadata': '3.1.1', 'playwright': '0.9.0'}, 'JAVA_HOME': 'C:\\Program Files\\Java\\jdk-22', 'Base URL': ''}
rootdir: C:\Users\Impossible\geekstore-automation-playwright
configfile: pytest.ini
testpaths: tests
plugins: base-url-2.1.0, html-4.2.0, metadata-3.1.1, playwright-0.9.0
collected 1 item

tests/test_login.py::TestLogin::test_open_browser[chromium] PASSED [100%]

----- Generated html report: file:///C:/Users/Impossible/geekstore-automation-playwright/reports/report.html -----
===== 1 passed in 3.39s =====
```

Ejemplo donde utilizar “pytest” dentro del venv utiliza test_login.py, e indica que en este caso la prueba paso.

Acerca de test_login.py

Para test_login.py, el caso de prueba está identificado como **TC001** y se encuentra marcado como prueba smoke y regression.

```
import pytest
from pages.login_page import LoginPage

@pytest.mark.smoke
```

```

@pytest.mark.regression
class TestLogin:
    """Tests para la funcionalidad de login"""

    def test_open_browser(self, page):
        """TC001: Navegar hacia pagina Geek Store"""
        login_page = LoginPage(page)
        login_page.navigate()

        # Validate we're on products page
        assert "/index.php" in page.url
        print("TC001: Navegación completada")

```

Primero se importa Pytest y la clase LoginPage. Los decoradores `@pytest.mark.smoke` y `@pytest.mark.regression` clasifican la prueba en los grupos smoke y regression, lo que permite ejecutar únicamente determinados grupos de pruebas utilizando los marcadores de Pytest.

El método `test_open_browser()` recibe `page`, que Playwright utiliza para controlar la página del navegador. Posteriormente, se crea un objeto de tipo `LoginPage` mediante `login_page = LoginPage(page)` y se llama a su método `navigate()` con `login_page.navigate()`.

Finalmente, se realiza la validación principal mediante `assert "/index.php" in page.url`, que comprueba que la URL actual contiene `/index.php`.

```

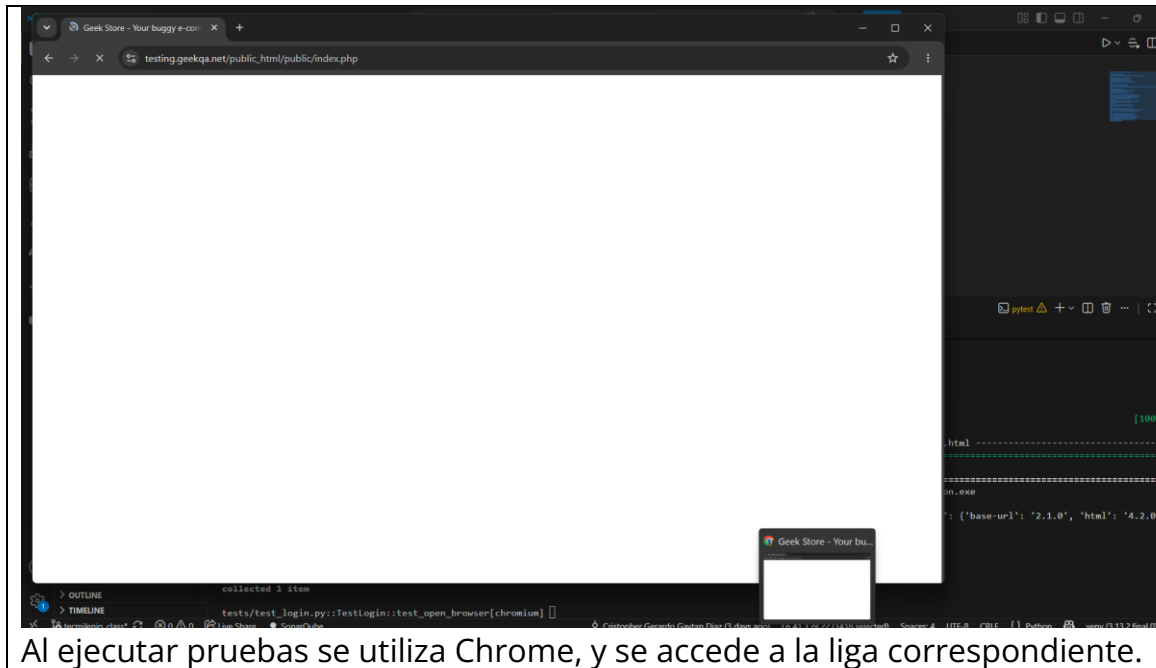
• (venv) PS C:\Users\Impossible\geekstore-automation-playwright> pytest -m smoke
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Impossible\geekstore-automation-playwright\venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.13.2', 'Platform': 'Windows-11-10.0.22631-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'base-url': '2.1.0', 'html': '4.2.0', 'metadata': '3.1.1', 'playwright': '0.9.0'}, 'JAVA_HOME': 'C:\\Program Files\\Java\\jdk-22', 'Base URL': ''}
rootdir: C:\Users\Impossible\geekstore-automation-playwright
configfile: pytest.ini
testpaths: tests
plugins: base-url-2.1.0, html-4.2.0, metadata-3.1.1, playwright-0.9.0
collected 1 item

tests/test_login.py::TestLogin::test_open_browser[chromium] PASSED [100%]

----- Generated html report: file:///C:/Users/Impossible/geekstore-automation-playwright/reports/report.html -----
===== 1 passed in 2.56s =====

```

Si se ejecutan pruebas de solo tipo “smoke”, se seguirá utilizando `test_login.py`, ya que incluye ese marcador.



Resultados

Las pruebas fueron ejecutadas correctamente y finalizaron con resultado satisfactorio. Durante la ejecución, Playwright abrió Chromium, accedió al sitio GeekStore y posteriormente Pytest realizó la validación establecida en el caso de prueba **TC001**. La condición de la prueba se cumplió al comprobar que la URL contenía /index.php, por lo que el caso de prueba fue marcado como exitoso.

Conclusiones

El proyecto permite conocer el funcionamiento básico de la automatización de pruebas web utilizando Python, Pytest y Playwright. La implementación actual demuestra cómo un caso de prueba puede controlar un navegador automáticamente, navegar hacia una página y comprobar una condición específica mediante una validación (assert).

Además, el uso del patrón Page Object Model permite organizar el código y facilita agregar nuevas pruebas y funcionalidades posteriormente, como pruebas de inicio de sesión, validación de mensajes de error y comprobación de diferentes comportamientos del sitio.